

CS3103 Project B Report

Group Information

Member	Name	SID	Email
#1 (contact person)	MURATOV Ali	57317500	amuratov4-c@my.cityu.edu.hk
#2	WANG Shenglin	57854016	swang586-c@my.cityu.edu.hk
#3	ALI Syed Ayan	57278309	saali6-c@my.cityu.edu.hk

Note: Please specify each team member's contribution if not all members make significant contributions to this project.

Problem 1

1) Have you successfully implemented the priority scheduling into the basekernel? Have your test programs executed properly? If not, please provide potential reasons for the issues.

Yes, we have implemented the priority scheduling into the basekernel.

2) Abstract idea and mechanism design.

1. Initialization: schedulertest.c

In the start up program already given, we have made improvements. Inside the main function, two arrays are defined, 'exec' for executing processes and 'pr' for storing priorities. The priority is passed as an argument to the syscall_process_run function.

Remark: we have five processes, so the "for loop" will execute five times.

2. Creating process: syscall_handler.c

In the sys_process_run function, create a process with a priority through the process_create_with_priority (int priority) function. For the parent class process, which is automatically created by the system, we set its priority to 0 through process_create(). In process_create_with_priority(), we define a variable(argc) to distinguish if it is a child process or parent process. Next, if the value is 1 or less, that means it is the parent process created by the system, the process is added directly to the readylist via process_launch(). If the value is greater than 1, that means the process is the child process, then the list_push_priority function in process_wait_p(process *p) is called to add it to the waiting list.

3. Implementing priority: list.c.

list_push_priority applies the ordering principle of data structures to add processes to the waiting list in order of priority. After all processes are sorted in the waiting list.

4. Executing processes: schedulertest.c

syscall_process_wakeup(), wakes up the process in the waiting list and executes it in FIFO order.

For detailed function explanations, see the comments in the code.

The problem encountered when solving problem 1:

Our solution requires the parent process (schedulertest.exe) to be run and child processes to be put into the waiting queue. In `sys_process_run()` we differentiated this two functions by `pid`. Our initial condition was: if (`pid == 2`) (meaning that it is the parent process) then do not put the process to the waiting queue. However, this is not optimal solution, because we assume that the schedulertest.exe can be run multiple times. When it is run for the second time, its `pid` changes. To account for that change, we had to implement different logic. In our final solution, we differentiate between parent and child processes by the “`argc`” argument’s value. For child processes, `argc = 2`, and for the parent process `argc <= 1`.

3) Implemented functions.

- schedulertest.c
 - `int syscall_process_run(pfd,argc,argv)`
 - `pfd`: process file descriptor
 - `argc`: argument count
 - `argv`: array with process.exe and priority
 - output: `pid` of process
 - `void syscall_process_wakeup()`
 - wake up the process in the waiting list ,and put them to the ready list to execute
- process.c:
 - `struct process *process_create():`
 - used to create the parent process with the priority = 0
 - output: process object
 - `struct process * process_create_with_priority(int pr):`
 - `int pr` : priority
 - create child process and assign the specified priority
 - output: process object
 - `void process_wait_p(struct process *p):`
 - `p`: child process
 - put the child process into the waiting list
 - `void process_wakeup_waiting():`
 - wake up the child processes in their order in the waiting list and put them into the ready list to execute
- list.c:
 - `void list_push_priority(struct list*list, struct list_node *node,int pri)`
 - `struct list*list` : the list (such as waiting list , ready list)
 - `list_node *node`: the process that be pushed into the list
 - `int pri`: priority of the process
 - This function will put the process into the list by the process’s priority.It will sort the order of the process according to its priority.

1.Initialization

```

C scheduler.c
#include "library/syscalls.h"

int main() {
    const char *exec[5] = {"bin/process1.exe", "bin/process2.exe", "bin/process3.exe", "bin/process4.exe", "bin/process5.exe"};
    const char *pr[5] = {"9", "7", "2", "1", "5"};

    for (int i = 0; i < 5; i++) {
        const char *argv[2];
        argv[0] = exec[i];
        argv[1] = pr[i];
        int argc = 2;
        int pfd = syscall_open_file(KNO_STDDIR, exec[i], 0, 0);
        if (pfd >= 0) {
            int pid = syscall_process_run(pfd, argc, argv);
            if (pid > 0) {
                // printf("started processsss %d\n", pid);
            } else {
                printf("couldn't run %s: %s\n", argv[0], strerror(-pid));
            }
            syscall_object_close(pfd);
        } else {
            printf("couldn't find %s: %s\n", argv[0], strerror(-pfd));
        }
    }

    for (int i = 0; i < 5; i++) {
        syscall_process_wakeup();
    }
}

```

Process

priority

Initialization

Put them to the ready list and execute

2.Creating Process

```

int sys_process_run( int fd, int argc, const char **argv)
{
    if(!is_valid_object_type(fd, KOBJECT_FILE)) return ERROR_INVALID_OBJECT;

    struct kobject *k = current->ktable[fd];

    /* Copy argv into kernel memory. */
    char **copy_argv = argv_copy(argc, argv);

    //xxx
    int priority = atoi(copy_argv[1]);

    /* Create the child process */
    struct process *p = process_create_with_priority(priority);
    process_inherit(current, p);

    /* SWITCH TO ADDRESS SPACE OF CHILD PROCESS */
    struct pagetable *old_pagetable = current->pagetable;
    current->pagetable = p->pagetable;
    pagetable_load(p->pagetable);

    /* Attempt to load the program image. */
    addr_t entry;
    int r = elf_load(p, k->data.file, &entry);
    if(r >= 0) {
        /* If load succeeded, reset stack and pass arguments */
        process_stack_reset(p, PAGE_SIZE);
        process_kstack_reset(p, entry);
        process_pass_arguments(p, argc, copy_argv);
    }

    /* SWITCH BACK TO ADDRESS SPACE OF PARENT PROCESS */
    current->pagetable = old_pagetable;
    pagetable_load(old_pagetable);

    /* Delete the argument and path copies. */
    argv_delete(argc, copy_argv);

    /* If any error happened, return in the context of the parent */
    if(r < 0) {
        if(r == ERROR_EXECUTION_FAILED) {
            process_delete(p);
        }
        return r;
    }

    /* Otherwise, launch the new child process. */
    if (argc > 1) {
        process_wait_p(p);
    } else {
        process_launch(p);
    }
    return p->pid;
}

int sys_process_wakeup()
{
    process_wakeup_waiting();
    return 0;
}

/* Function creates a child process with the standard window replaced by wd */
int sys_process_wrun( int fd, int argc, const char **argv, int *fds, int fd_len)
{
    if(!is_valid_object_type(fd, KOBJECT_FILE)) return ERROR_INVALID_OBJECT;
    struct kobject *k = current->ktable[fd];
}

```

Child Process: Add to the waiting list

Parent Process: Launch it directly.

```

struct process *process_create_with_priority(int pr)
{
    struct process *p;

    p = page_alloc(1);

    p->pid = process_allocate_pid();
    process_table[p->pid] = p;

    p->pagetable = pagetable_create();
    pagetable_init(p->pagetable);

    p->vm_data_size = 0;
    p->vm_stack_size = 0;

    process_data_size_set(p, 2 * PAGE_SIZE);
    process_stack_size_set(p, 2 * PAGE_SIZE);

    p->kstack = page_alloc(1);
    p->kstack_top = p->kstack + PAGE_SIZE - 8;
    p->kstack_ptr = p->kstack_top - sizeof(struct x86_stack);

    process_kstack_reset(p, PROCESS_ENTRY_POINT);

    // XXX table should be allocated
    int i;
    for(i = 0; i < PROCESS_MAX_OBJECTS; i++) {
        p->ktable[i] = 0;
    }

    p->state = PROCESS_STATE_READY;
    p->priority = pr;
}

```

3.Implementing Priority

```
7
5 void list_push_priority(struct list *list, struct list_node *node, int pri)
6 {
7     struct list_node *n;
8     int i = 0;
9     if(!list->head) {
10         list_push_head_pri(list, node, pri);
11         return;
12     }
13     for(n = list->head; n; n = n->next) {
14         if(pri > n->priority || i > 5000) {
15             node->next = n;
16             node->prev = n->prev;
17             node->priority = pri;
18             if(n->prev) {
19                 n->prev->next = node;
20             } else {
21                 list->head = node;
22             }
23             n->prev = node;
24             node->list = list;
25             list->size++;
26             return;
27         }
28         i++;
29     }
30     list_push_tail_pri(list, node, pri);
31 }
32
```

Sorting the process by priority

4.Executing Process

```
for (int i = 0; i < 4; i++) {
    syscall_process_wakeup();
}
```

```
1 int sys_process_wakeup()
2 {
```

```
3     process_wakeup_waiting();
```

```
4     return 0;
5 }
```

```
5
7 /* Function creates a child process with the standard
```

```
358
359 void process_wakeup_waiting() {
360     struct process *p;
361     p = (struct process *) list_pop_head(&waiting_list);
362     if(p) {
363         printf("Priority: %d\n", p->priority);
364         p->state = PROCESS_STATE_READY;
365         list_push_head(&ready_list, &p->node);
366     }
367 }
```

Put them to the ready list to execute

Problem 1 Execution Procedure:

In linux Terminal:

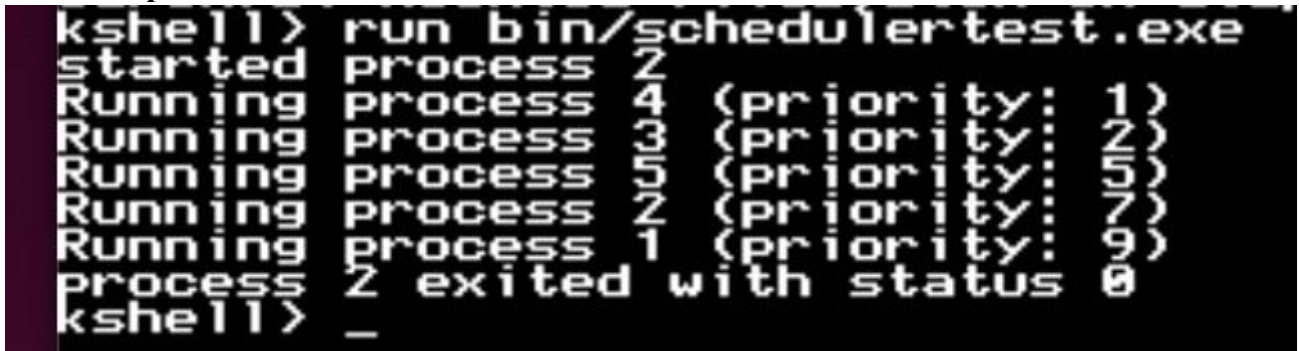
1. cd basekernel
2. export path correctly
3. make clean
4. make
5. qemu-system-i386 -cdrom basekernel.iso

Then In Qemu:

1. automount
2. run bin/schedulertest.exe

priority: process1: 9; process2: 7; process3: 2; process4: 1; process5: 5

Qemu Output:



```
kshell> run bin/schedulertest.exe
started process 2
Running process 4 (priority: 1)
Running process 3 (priority: 2)
Running process 5 (priority: 5)
Running process 2 (priority: 7)
Running process 1 (priority: 9)
process 2 exited with status 0
kshell> _
```

Problem 2

1) Have you successfully implemented the named pipe into the basekernel? Have your test programs executed properly? If not, please provide potential reasons for the issues.

Yes, we have successfully implemented the named pipe into the basekernel, and the test programs execute properly.

2) Abstract idea and mechanism design.

Algorithm:

1. Sender creates a file in the specified directory
 - a. if provided path leads to an existing directory, the file is created in that directory using `fs_dirent_mkfile()`
 - b. if provided path does not lead to an existing directory, the provided directory is created using `fs_dirent_mkdir()` and file is created within the new folder using `fs_dirent_mkfile()`
 - c. created file is not added to the receiver's and sender's kernel tables at this step
2. Sender tries to open the file named `filename` at the path specified by `pathname`
 - a. Sender checks if a named pipe corresponding to the file named `filename` exists in the its kernel table

- i. This check is performed by iterating through each kernel object in the sender's (i.e. local) kernel table and checking if the `named_pipe` of that kernel object contains the name equal to the `filename`
 - b. If such named pipe already exists, then its file descriptor is returned
 - c. If no such named pipe is found, then a new named pipe is created and put into the kernel table. To create a new named pipe, the following steps are taken:
 - i. File is retrieved from the filesystem using `fs_resolve()`
 - ii. New named pipe object is created, its `name` and `f` fields are initialized to the retrieved file's name and `fs_dirent` reference respectively
 - iii. Created named pipe object is put inside kernel object
 - iv. Kernel object is then put inside kernel table, its file descriptor is returned
3. Sender writes to the file
 - a. The file is retrieved using the file descriptor returned from the function which opened the file
 - i. File descriptor is used to locate the kernel object containing named pipe object corresponding to the file of interest
 - ii. By unpacking the named pipe object, we get access to the `fs_dirent` object of the file
 - iii. `fs_dirent` object is passed to `fs_dirent_write()` function which writes the byte stream to the file
4. Since filesystem is shared between unrelated processes, receiver process is able to access that file
5. Receiver opens the file according to the specified path
 - a. same as (2)
6. Receiver reads the file
 - a. The file is retrieved using the file descriptor returned from the function which opened the file
 - i. File descriptor is used to locate the kernel object (in the receiver's kernel table) containing named pipe object corresponding to the file of interest
 - ii. By unpacking the named pipe object, we get access to the `fs_dirent` object of the file
 - iii. `fs_dirent` object is passed to `fs_dirent_write()` function which writes the byte stream to the file

Key idea: Receiver and Sender must have their own local instances of kernel objects containing `'fs_dirent'` references to the files they open. It is important to note that these references must point to the file object. This ensures that the offsets for "read" and "write" operations are updated properly. This is important for the following reasons:

- The functions `fs_dirent_read()` and `fs_dirent_write()` update the offset of the kernel object whenever a document is read from or written to. If we assume that the sender and receiver use the same kernel object (for example, if we used a global array of kernel objects accessible both by sender and receiver, or if we assumed that the kernel table is shared between sender and receiver), then every write operation will update the kernel object's offset value, causing the next read operation to start reading from the position where the writing operation was ended. This logic is incorrect, and we avoid it.

Difficulties faced:

- Initially, we thought that the kernel table is shared between two unrelated processes. Since this is not true, our initial implementation did not work correctly. After that, we decided to avoid using global data structures and came to the solution described above.
- To run two processes, we used the solution for the problem 1. We assigned higher priority for the sender process because it must be run before the receiver process.

3) Implemented functions.

For the explanation of these functions, please refer to the comments in the code and algorithm.

- `syscall_handler.c`
 - `int sys_make_named_pipe(const char *pathname, const char *filename)`
 - purpose: Creates a file at the specified path (creates the folder specified in "pathname" if it does not exist)
 - pathname: the name of the folder where you want to create the named pipe's file
 - filename: the name of the named pipe's file
 - output: status code
 - returns 1 if operation was completed successfully
 - returns -1 if some error happened
 - `int sys_open_named_pipe(const char *pathname, const char* filename)`
 - purpose: Retrieves kernel object corresponding to the file specified by the pathname and filename
 - pathname: the name of the folder where the file that you want to open is located
 - filename: the name of the named pipe's file
 - output: file descriptor of the kobject which contains the named_pipe object or -1 if fd was not retrieved successfully
- `kobject.c`
 - `struct kobject *kobject_create_named_pipe(struct named_pipe *p)`
 - purpose: Creates a kernel object encapsulating the named_pipe that is passed as an argument
 - p: the named_pipe object for which you want to create a kernel object
 - output: kernel object which contains reference for the named_pipe
- `named_pipe.c`
 - `struct named_pipe *named_pipe_create(struct fs_dirent *file, const char *filename)`
 - purpose: create a named_pipe object which will contain the reference to the file and the file's name
 - file: reference for the file which will be stored inside named pipe
 - filename: the name of the file which named pipe will also store
 - output: named pipe object corresponding to the file named as "filename" as referenced by "file"
 - `int named_pipe_read(struct named_pipe *p, char *buffer, uint32_t length, uint32_t offset)`
 - purpose: Reads from the file referenced in the attribute
 - p: a named pipe object which contains the file from which we want to read data
 - buffer: an array of characters from where the data read from the file will be stored

- length: the size of the buffer
- offset: position in the file where to start to read
- output: number of bytes read
- `int named_pipe_write(struct named_pipe *p, const char *buffer, uint32_t length, uint32_t offset)`
 - purpose: Writes to the file referenced in the attribute
 - p: a named pipe object which contains the file where we want to write data
 - buffer: an array of characters which will be written to the file
 - length: the size of the buffer
 - offset: position in the file where to start to write
 - output: number of bytes written

Steps to run:

- in the terminal:
- `cd basekernel`
- export path correctly
 - the version that I use is: `export PATH="$HOME/Desktop/basekernel/cross/bin:$PATH"`
 - if you place the folder in another directory, please adjust the path accordingly
- `make clean`
- `make run`
- in the qemu simulator:
 - mount ata 0 diskfs
 - install atapi 2 ata 0
 - list (until new blank page appears in the qemu simulator, allowing all outputs to be seen when the test program runs)
 - run `bin/test_problem2.exe`

the result should look like this:

```

kshell> run bin/test_problem2.exe
started process 2

Creating a new file...
File successfully created

Sender:
Opening the named pipe...
Named pipe opened successfully
Sending byte stream: Hello World
Bytes written: 12

Receiver:
Opening the named pipe...
Named pipe opened successfully
Bytes read: 12
Byte stream read: Hello World

process 2 exited with status 0
kshell> $_

```

Steps to create new test cases:

1. Change pathname to the name of the folder where you want to create your new file. This folder may already exist or not exist. If it does not exist, the program will create a new folder

and put it in the current working directory (image folder). Do not add slashes. Slash is added automatically when 'concatenate_chars()' function is run.

2. Change the filename to name the file as you want. Do not add slashes.
3. Change the message contained in the buffer to the one that you want.

High-level diagram depicting hierarchy of objects in our solution:

